Data Transformation

What is data transformation?

Data transformation is the process of changing the format, structure, or values of data. For data analytics projects, data may be transformed at two stages of the data pipeline. Organizations that use on-premises data warehouses generally use an ETL (extract, transform, load) process, in which data transformation is the middle step. Today, most organizations use cloud-based data warehouses, which can scale compute and storage resources with latency measured in seconds or minutes. The scalability of the cloud platform lets organizations skip preload transformations and load raw data into the data warehouse, then transform it at query time — a model called ELT (extract, load, transform).

Processes such as data integration, data migration, data warehousing, and data wrangling all may involve data transformation.

Data transformation may be constructive (adding, copying, and replicating data), destructive (deleting fields and records), aesthetic (standardizing salutations or street names), or structural (renaming, moving, and combining columns in a database).

An enterprise can choose among a variety of ETL tools that automate the process of data transformation. Data analysts, data engineers, and data scientists also transform data using scripting languages such as Python or domain-specific languages like SQL.

Benefits and challenges of data transformation

- Data is transformed to make it better-organized. Transformed data may be easier for both humans and computers to use.
- Properly formatted and validated data improves data quality and protects applications from potential landmines such as null values, unexpected duplicates, incorrect indexing, and incompatible formats.
- Data transformation facilitates compatibility between applications, systems, and types of data. Data used for multiple purposes may need to be transformed in different ways..

  However, there are challenges to transforming data effectively:
- Data transformation can be expensive. The cost is dependent on the specific infrastructure, software, and tools used to process data. Expenses may include those related to licensing, computing resources, and hiring necessary personnel.

- Data transformation processes can be resource-intensive. Performing transformations in an on-premises data warehouse after loading, or transforming data before feeding it into applications, can create a computational burden that slows down other operations. If you use a cloud-based data warehouse, you can do the transformations after loading because the platform can scale up to meet demand.
- Lack of expertise and carelessness can introduce problems during transformation. Data analysts without appropriate subject matter expertise are less likely to notice typos or incorrect data because they are less familiar with the range of accurate and permissible values. For example, someone working on medical data who is unfamiliar with relevant terms might fail to flag disease names that should be mapped to a singular value or notice misspellings.
- Enterprises can perform transformations that don't suit their needs. A business might change information to a specific format for one application only to then revert the information back to its prior format for a different application.

Introduction to Feature Scaling

While working with datasets from ML projects that had multiple features spanning varying degrees of magnitude, range, and units. This is a significant obstacle as a few machine learning algorithms are highly sensitive to these features.

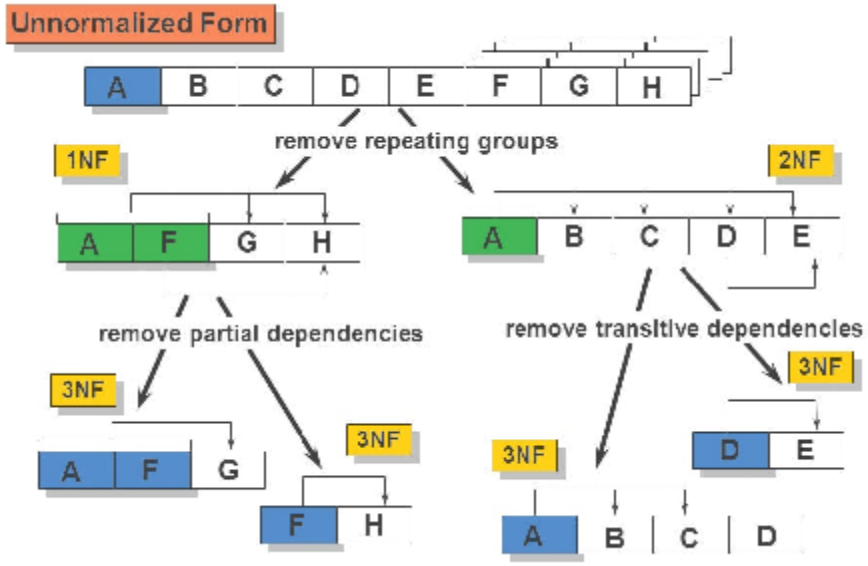
Most of us must have faced this issue in our projects or our learning journey. For example, one feature is entirely in kilograms while the other is in grams, another one is liters, and so on. How can we use these features when they vary so vastly in terms of what they're presenting?

This is where I turned to the concept of feature scaling. It's a crucial part of the data preprocessing stage but I've seen a lot of beginners overlook it (to the detriment of their machine learning model).

Data normalization

Normalization is a scaling technique in which values are shifted and rescaled so that they end up ranging between 0 and 1. It is also known as Min-Max scaling.

The data normalization (also referred to as data pre-processing) is a basic element of data mining. It means transforming the data, namely converting the source data in to another format that allows processing data effectively. The main purpose of data normalization is to minimize or even exclude duplicated data



What is data mining normalization?

The data normalization (also referred to as data pre-processing) is a basic element of data mining. It means transforming the data, namely converting the source data in to another format that allows processing data effectively. The main purpose of data normalization is to minimize or even exclude duplicated data. This is a very essential and important issue because it is increasingly problematic to keep in data in relational databases, which store identical data in more than one place.

The use of data mining normalization has a number of advantages:

- The application of data mining algorithms becomes easier
- The data mining algorithms get more effective and efficient
- The data is converted in to the format that everyone can get their heads around
- The data can be extracted from databases faster
- It is possible to analyze the data in a specific manner

As you can see applying data mining normalization is reasonable and allows achieving certain results. There are some data mining normalization techniques that are widely used for the data transformation and which will be discussed below.

Min-Max normalization

The first technique we will cover is min-max normalization. It is the linear transformation of the original unstructured data. It scales the data from 0 to 1. It is calculated by the following formula:

$$v' = \frac{v - \text{minF}}{\text{maxF} - \text{minF}} (new_max_F - new_min_F) + new_min_F ,$$

where is the current value of feature *F*.

Let us consider one example to make the calculation method clear. Assume that the minimum and maximum values for the feature F are $50,000 and $100,000 correspondingly. It needs to range *F* from 0 to 1. In accordance with min-max normalization, *v* = $80,000 is transformed to:

$$v' = \frac{80{,}000 - 50{,}000}{100{,}000 - 50{,}000} + (1 - 0) + 0 = \frac{3}{5} = 0{,}6$$

As you can see this technique enables to interpret the data easily. There are no large numbers, only concise data that do not require further transformation and can be used in decision-making process immediately.

Z-score normalization

The next technique is z-score normalization. It is also called zero-mean normalization. The essence of this technique is the data transformation by the values conversation to a common scale where an average number equals zero and a standard deviation is one. A value is normalized to ′ under the formula:

$$v' = \frac{\mathrm{v} - \overline{F}}{\sigma_F},$$

Here is the mean and is the standard deviation of feature *F*.

Here is an example of the calculation of a value.

On the supposition that the mean of feature is $65,000 and its standard deviation is $ 18,000. Applying the z-score normalization we get the following mean of the value equals to $85,800:

$$\frac{85{,}800 - 65{,}000}{18{,}000} = 1{,}156.$$

Therefore, z-score normalization is an approach to the data processing that takes into account outliers; however, it does not provide data normalization with an identical scale.

Data normalization by decimal scaling

And now we finally will move on to the decimal scaling normalization technique. It involves the data transformation by dragging the decimal points of values of feature *F*. The movement of decimals is very dependent on the absolute value of the maximum. A value of feature *F* is transformed to by calculating:

$$v' = \frac{v}{10^j}.$$

In this formula, *j* is the lowest integer while *Max* (|) < 1.

For instance, values for feature *F* scale from 850 to 825. Suppose *j* equals to three. In this case, the maximum absolute value of feature *F* equals 850. Applying the normalization with decimal scaling, we need to divide all values by 1,000. Therefore, we get 850 normalized to 0,850 as well as 825 transformed to 0,825.

This technique entails the transformation of the decimal points of the values according to the absolute value of the maximum. It follows that the means of the normalized data will always be between 0 and 1.

Standardization

Standardization is another scaling technique where the values are centered around the mean with a unit standard deviation. This means that the mean of the attribute becomes zero and the resultant distribution has a unit standard deviation.

The result of standardization (or Z-score normalization) is that the features will be rescaled to ensure the mean and the standard deviation are 0 and 1, respectively. The equation is shown below:

$$x_{\text{stand}} = \frac{x - \text{mean}(x)}{\text{standard deviation } (x)}$$

This technique to rescale features value with the distribution value between 0 and 1 is useful for the optimization algorithms, such as gradient descent, that are used within machine-learning

algorithms that weight inputs (e.g., regression and neural networks). Rescaling is also used for algorithms that use distance measurements, for example, K-nearest-neighbours (KNN).

The Big Question – Normalize or Standardize?

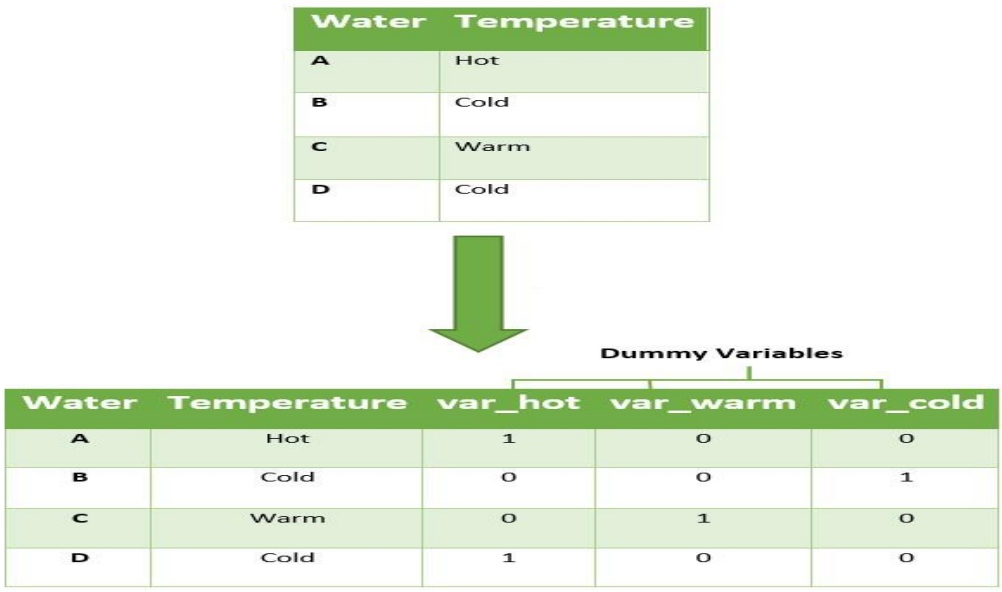
Normalization vs. standardization is an eternal question among machine learning newcomers. Let me elaborate on the answer in this section.

- Normalization is good to use when you know that the distribution of your data does not follow a Gaussian distribution. This can be useful in algorithms that do not assume any distribution of the data like K-Nearest Neighbors and Neural Networks.
- Standardization, on the other hand, can be helpful in cases where the data follows a Gaussian distribution. However, this does not have to be necessarily true. Also, unlike normalization, standardization does not have a bounding range. So, even if you have outliers in your data, they will not be affected by standardization.

Binary Transformation

Create Dummy Variables in Python with Pandas

A dataset may contain various type of values, sometimes it consists of categorical values. So, in-order to use those categorical value for programming efficiently we create dummy variables. A dummy variable is a binary variable that indicates whether a separate categorical variable takes on a specific value.



As we can see three dummy variables are created for the three categorical values of the temperature attribute. We can create dummy variables in python using *get_dummies()* method.

Syntax: *pandas.get_dummies(data, prefix=None, prefix_sep='_',)*

Parameters:

- *data= input data i.e. it includes pandas data frame. list . set . numpy arrays etc.*
- *prefix= Initial value*
- *prefix_sep= Data values separation.*

Return Type: *Dummy variables.*

```
# import required modules
import pandas as pd
import numpy as np

# create dataset
df = pd.DataFrame({'Temperature': ['Hot', 'Cold', 'Warm',
'Cold'],})

# display dataset
print(df)
```

```
   Temperature
0          Hot
1         Cold
2         Warm
3         Cold
```

```
# create dummy variables
pd.get_dummies(df)
```

	Temperature_Cold	Temperature_Hot	Temperature_Warm
0	0	1	0
1	1	0	0
2	0	0	1
3	1	0	0

```
from matplotlib import pyplot as plt
import pandas as pd
import numpy as np
import warnings
warnings.filterwarnings('ignore')

dfr=pd.read_csv('result.csv')
dfr
```

	Reg No	Name	M1	M2	M3	Result
0	176CS21001	Anil	90	98	95	Distinction
1	176CS21002	Sangamesh	75	72	69	Distinction
2	176CS21003	Pooja	65	66	68	First
3	176CS21004	Jayashree	58	50	56	Second
4	176CS21706	Vinod	43	44	50	Second
5	176CS21707	Ravi	35	30	28	Fail

```
dfr['Result'].unique()
```

```
array(['Distinction', 'First', 'Second', 'Fail'], dtype=object)
```

```
pd.get_dummies(dfr['Result'])
```

	Distinction	Fail	First	Second
0	1	0	0	0
1	1	0	0	0
2	0	0	1	0
3	0	0	0	1
4	0	0	0	1
5	0	1	0	0